Vidyavardhini's
College of Engineering & Technology

Vasai Road (W)

Department of
Artificial Intelligence and Data Science

Laboratory Manual : 2025-26
(Student Copy)

Semester	VIII	Class	B.E
Course Code	CSDOL8022		
Course Name	Recommendation Systems Lab		
Faculty Name	Mr. Raunak Joshi		

Vidyavardhini's College of Engineering & Technology

Vision

To be a premier institution of technical education; always aiming at becoming a valuable resource for industry and society.

Mission

- To provide a technologically inspiring environment for learning.
- To promote creativity, innovation and professional activities.
- To inculcate ethical and moral values.
- To cater personal, professional and societal needs through quality education.

Program Outcomes (POs):

Engineering Graduates will be able to:

- **PO1. Engineering knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified respectively to develop the solution of complex engineering problems.
- **PO2. Problem analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration of sustainable development.
- **PO3. Design/development of solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.
- **PO4. Conduct investigations of complex problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.
- **PO5. Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling, recognizing their limitations to solve complex engineering problems.
- **PO6. The Engineer and the World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.
- **PO7. Ethics:** Apply ethical principles and commit to professional ethics and human values, diversity and inclusion; adhere to national & international laws.
- **PO8. Individual and Collaborative Team Work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
- **PO9. Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
- **PO10. Project management and finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
- **PO11. Life-long learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Course Objectives

1	To understand the key concepts of Recommendation systems
2	Design and implement cluster-based approaches for recommendation systems
3	Design, implement and analyze classification algorithms for recommendation systems
4	To understand various Recommendation system Algorithms.
5	To understand data processing for Recommendation system Algorithms

Course Outcomes

CO	At the end of course students will be able to:	Action verbs	Bloom's Level
CSDOL8022.1	Apply linear algebra techniques for foundation of recommendation systems.	Apply	Apply (Level 3)
CSDOL8022.2	Apply Collaborative filtering techniques for recommendation systems	Apply	Apply (Level 3)
CSDOL8022.3	Apply Content-based techniques for recommendation systems.	Apply	Apply (Level 3)
CSDOL8022.4	Apply Knowledge-based representations and techniques for recommendation systems.	Apply	Apply (Level 3)
CSDOL8022.5	Applying feature engineering and pre-processing for recommendation systems.	Apply	Apply (Level 3)
CSDOL8022.6	Design real world problems using recommendation systems.	Design	Create (Level 6)

Mapping of Experiments with Course Outcomes

List of Experiments	Course Outcomes					
	CSDOL 8022.1	CSDOL 8022.2	CSDOL 8022.3	CSDOL 8022.4	CSDOL 8022.5	CSDOL 8022.6
Implementation of Matrix operations and data representation towards understanding mathematics for recommendation system	3	-	-	-	-	-
Implementation of Feature engineering and pre-processing of data for recommendation systems	-	-	-	-	3	-
Implementation of Bayes classifier for recommendation	-	-	3	-	-	-
Implement User-based Nearest neighbor recommendation	-	3	-	-	-	-
Implement Item-based Nearest neighbor recommendation	-	3	-	-	-	-
Implement Content-based recommendation system	-	-	3	-	-	-
Implementation of a recommendation system using Hybrid approach	-	-	-	-	-	3
Implementation of a Regression based recommendation system	-	-	-	3	-	-

INDEX

Sr. No.	Name of Experiment	D.O.P.	D.O.C.	Page No.	Remark
1	Implementation of Matrix operations and data representation towards understanding mathematics for recommendation system				
2	Implementation of Feature engineering and pre-processing of data for recommendation systems				
3	Implementation of Bayes classifier for recommendation				
4	Implement User-based Nearest neighbor recommendation				
5	Implement Item-based Nearest neighbor recommendation				
6	Implement Content-based recommendation system				
7	Implementation of a recommendation system using Hybrid approach				
8	Implementation of a Regression based recommendation system				

D.O.P: Date of performance

D.O.C : Date of correction

Experiment No. 1
Implementation of Matrix operations and data representation towards understanding mathematics for recommendation system
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implementation of Matrix operations and data representation towards understanding mathematics for recommendation system

Objective: Ability to understand the different matrix operations performed using Python respectively.

Theory: The matrix operations help us to combine two or more matrices, to form a single matrix. The arithmetic operations of addition, subtraction, multiplication can also be performed on matrices. Furthermore, sometimes matrix operations can change a matrix itself. The operation of the transpose of a matrix, the inverse of a matrix, helps to transform a specific matrix onto itself. The following are some of the important matrix operations. Addition - matrix operations, Subtraction - matrix operations, Multiplication - matrix operations, Transpose operation of a matrix and Inverse operation of a matrix.

Addition of Matrices

As we add two numbers we can easily add two matrices. The only thing we have to note is that the order of both the matrices that are to be added must be the same. That is to add two matrices we have to make sure that they are of the same order and then each element of the first matrix adds with each element of the second matrix to get a single matrix and thus the addition operation gets completed.

Subtraction of Matrices

As we add two matrices we can also easily subtract two matrices. The only thing we have to note is that the order of both the matrices that are to be subtracted must be the same. That is to subtract two matrices we have to make sure that they are of the same order and then each element of the first matrix is subtracted with each element of the second matrix to get a single matrix and thus the subtraction operation gets completed.

Scalar Multiplication of Matrices

For any matrix A = [aij]m×n if we multiply the matrix A with any scaler (say k) then the scaler is multiplied by each element of the matrix and this is called the scalar multiplication of matrices.

Multiplication of Matrix

Matrix multiplication is the operation that helps us to multiply two matrices. This is different from algebraic multiplication and not all the matrices can be multiplied. Only those matrices can be multiplied where the number of columns in the first is equal to the number of rows in the second, i.e for matrix Am×n and matrix Bn×p the multiplication is possible for any other matrices where the column of the first matrix is not equal to the row in the second matrix the multiplication is not possible.

Transpose Operation of a Matrix

Transpose operation of a matrix is used to find the transpose of any matrix. Transpose of any matrix is a matrix in which the rows of the matrix are changed to the column of the matrix and the column of the matrix is changed to the rows of the matrix. Suppose we have a matrix A of order m×n such that A =[ij]m×n then the transpose of matrix A is represented as (A)T

Inverse Operation of a Matrix

For any matrix A its inverse is found only when A is a square matrix and its determinant is equal to 1.

Implementation:

```
import numpy as np
```

```
# Define matrices

A = np.array([[1, 2, 3],

              [4, 5, 6]])


B = np.array([[7, 8, 9],

              [1, 2, 3]])


C = np.array([[1, 2],

              [3, 4],

              [5, 6]])


# 1. Addition of Matrices

print("Addition of A and B:")

print(A + B)


# 2. Subtraction of Matrices

print("\nSubtraction of A and B:")

print(A - B)


# 3. Scalar Multiplication

k = 2

print("\nScalar Multiplication of A by 2:")
```

```
print(k * A)


# 4. Matrix Multiplication

print("\nMatrix Multiplication of A and C:")

print(np.dot(A, C))


# 5. Transpose of Matrix

print("\nTranspose of Matrix A:")

print(A.T)


# 6. Inverse of Matrix

D = np.array([[1, 2],

              [3, 4]])


print("\nMatrix D:")

print(D)


print("\nInverse of Matrix D:")

print(np.linalg.inv(D))
```

Conclusion:

In this experiment, various matrix operations such as addition, subtraction, scalar multiplication, matrix multiplication, transpose, and inverse were successfully implemented using Python and NumPy. These operations are fundamental in understanding how data can be represented and manipulated mathematically in recommendation systems. Matrix operations play a crucial role in handling user-item interactions, transforming data, and optimizing algorithms. Hence, mastering these operations provides a strong foundation for building efficient and scalable recommendation systems.

Experiment No. 2
Implementation of Feature engineering and pre-processing of data for recommendation systems
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implementation of Feature engineering and pre-processing of data for recommendation systems.

Objective: Understanding the feature engineering and feature scaling and applying pre-processing techniques for recommendations.

Theory:

Preprocessing data is an important step in any data analysis task. It is usually the first step in any data mining or machine learning project. The aim of preprocessing is to clean the data, remove any noise and unwanted information, and prepare the data for further analysis. Feature engineering is the process of creating new features from existing data.
This can be done by transforming existing features, or by combining multiple features to create a new one. Feature engineering is a powerful tool that can help to improve the accuracy of machine learning models. In this guide, we will take a look at some of the most common preprocessing and feature engineering techniques.
Data preprocessing is an essential step in any data science or machine learning project. It is the process of cleaning and preparing the data for analysis. This step is important because it can help improve the accuracy of the results and make the data more manageable.

One of the greatest challenges in data analysis is dealing with missing data. When data is missing, it can be difficult to make accurate predictions. Data preprocessing can help deal with missing data by imputing missing values or using a technique called feature engineering.
Feature engineering is the process of creating new features from existing data. This can be done by transforming or combining existing features. For example, you could combine two features to create a new feature that is more predictive. Feature engineering can help improve the accuracy of machine learning models by making the data more representative of the real problem.
There are a few common tasks that data preprocessing typically performs:

- Identifying and handling missing values: This step is important in order to avoid bias in your results. Data preprocessing will identify missing values and either remove them or impute them with another value.
- Identifying and handling outliers: Outliers can often be found in data sets and can sometimes be caused by errors in data entry. Data preprocessing will identify outliers and either remove them or transform them so that they are no longer outliers.
- Feature engineering: This is the process of creating new features from existing data. This can be done by combining existing features, or by Transformations such as scaling or normalization.
- Data scaling and normalization: This step is important for many machine learning algorithms. Some algorithms require that the data be scaled or normalized in order to work correctly. Data preprocessing can take care of this for you.

Removing invalid values

Invalid values are values that are not valid for the data set. Invalid values can occur for a variety of reasons, such as errors in data entry or data that has been corrupted. Invalid values can cause problems with data modeling, so it is important to remove them from the data set.

Imputing missing values

Missing values are values that are not present in the data set. Missing values can occur for a variety of reasons, such as data that has been censored or data that has not been collected. Missing values can cause problems with data modeling, so it is important to impute them.

Scaling data

Scaling data is a method of normalizing data. Normalizing data is important for data modeling because it can help improve the accuracy of models. There are a variety of ways to scale data, but a simple method is to use the "scale()" function in Python. This function will scale the data so that the mean is 0 and the standard deviation is 1.

Implementation:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Sample dataset (User-Item Ratings)
```

```
data = {
    'user_id': [1, 2, 3, 4, 5],
    'age': [25, np.nan, 35, 45, 22],
    'income': [50000, 60000, np.nan, 80000, 45000],
    'rating': [5, 4, np.nan, 3, 5]
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)

# 1. Handling Missing Values (Imputation)
df['age'].fillna(df['age'].mean(), inplace=True)
df['income'].fillna(df['income'].mean(), inplace=True)
df['rating'].fillna(df['rating'].median(), inplace=True)

print("\nAfter Handling Missing Values:")
print(df)

# 2. Removing Invalid Values (Example: negative income)
df = df[df['income'] > 0]

# 3. Feature Engineering (Create new feature: income per age)
df['income_per_age'] = df['income'] / df['age']

print("\nAfter Feature Engineering:")
print(df)

# 4. Feature Scaling (Normalization)
scaler = StandardScaler()
scaled_features = scaler.fit_transform(df[['age', 'income', 'income_per_age']])
```

```
scaled_df = pd.DataFrame(scaled_features, columns=['age_scaled', 'income_scaled', 'income_per_age_scaled'])

print("\nScaled Features:")
print(scaled_df)

# Final Processed Data
final_df = pd.concat([df.reset_index(drop=True), scaled_df], axis=1)

print("\nFinal Preprocessed Dataset:")
print(final_df)
```

Conclusion:

In this experiment, data preprocessing and feature engineering techniques were successfully implemented using Python. Missing values were handled using imputation methods, and invalid data was removed to ensure data quality. New features were created to enhance the dataset, improving its representation for recommendation systems. Additionally, feature scaling was applied to normalize the data, which is essential for many machine learning algorithms. Overall, these preprocessing steps help in improving model accuracy, efficiency, and reliability in recommendation systems.

Experiment No. 3
Implementation of Bayes classifier for recommendation
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implementation of Bayes classifier for recommendation

Objective: Able to perform naïve bayes classifier for implementation recommendations system.

Theory: Naive Bayes classifiers are a collection of classification algorithms based on Bayes' Theorem. It is not a single algorithm but a family of algorithms where all of them share a common principle, i.e. every pair of features being classified is independent of each other. To start with, let us consider a dataset.

One of the most simple and effective classification algorithms, the Naïve Bayes classifier aids in the rapid development of machine learning models with rapid prediction capabilities.

The Naïve Bayes algorithm is used for classification problems. It is highly used in text classification. In text classification tasks, data contains high dimensions (as each word represents one feature in the data). It is used in spam filtering, sentiment detection, rating classification etc. The advantage of using naïve Bayes is its speed. It is fast and making predictions is easy with a high dimension of data.

Implementation:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score


# Sample dataset (User preferences for recommendation)

data = {

    'age': [25, 30, 22, 35, 40, 28, 50, 48],

    'income': [50000, 60000, 45000, 80000, 90000, 52000, 100000, 110000],
```

```
    'genre_action': [1, 0, 1, 0, 0, 1, 0, 0],

    'genre_comedy': [0, 1, 1, 0, 1, 0, 1, 0],

    'like_movie': [1, 1, 1, 0, 0, 1, 0, 0]  # Target (1 = Like, 0 = Not Like)

}


df = pd.DataFrame(data)


print("Dataset:")

print(df)


# Features and Target

X = df[['age', 'income', 'genre_action', 'genre_comedy']]

y = df['like_movie']


# Split data into training and testing

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# Apply Naive Bayes Classifier

model = GaussianNB()

model.fit(X_train, y_train)


# Predictions
```

```
y_pred = model.predict(X_test)


# Accuracy

accuracy = accuracy_score(y_test, y_pred)


print("\nPredictions:", y_pred)

print("Actual:", y_test.values)

print("Accuracy:", accuracy)


# Example: Recommend for a new user

new_user = [[27, 55000, 1, 0]]  # Example features

prediction = model.predict(new_user)


print("\nNew User Prediction (1=Like, 0=Not Like):", prediction[0])
```

Conclusion:

In this experiment, the Naïve Bayes classifier was implemented for a recommendation system using Python. A sample dataset representing user preferences was used to train the model. The Gaussian Naïve Bayes algorithm was applied to classify whether a user would like a particular item based on features such as age, income, and genre preferences. The model demonstrated efficient and fast prediction capability, making it suitable for recommendation tasks. Thus, Naïve Bayes proves to be a simple yet powerful technique for building scalable and effective recommendation systems.

Experiment No. 4
Implement User-based Nearest neighbor recommendation
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implement User-based Nearest neighbor recommendation

Objective: Ability to perform User based nearest neighbor recommendation.

Theory: User-Based Collaborative Filtering is a technique used to predict the items that a user might like on the basis of ratings given to that item by other users who have similar taste with that of the target user. Many websites use collaborative filtering for building their recommendation system. Steps for User-Based Collaborative Filtering:

Step 1: Finding the similarity of users to the target user U. Similarity for any two users 'a' and 'b' can be calculated from the given

$$Sim(a,b) = \frac{\sum_p (r_{ap}-\bar{r}_a)(r_{ab}-\bar{r}_b)}{\sqrt{\sum (r_{ap}-\bar{r}_a)^2}\sqrt{\sum (r_{bp}-\bar{r}_b)^2}}$$

$$r_{up} : rating\ of\ user\ u\ against\ item\ p$$

$$p : items$$

Step 2: Prediction of missing rating of an item Now, the target user might be very similar to some users and may not be much similar to others. Hence, the ratings given to a particular item by the more similar users should be given more weightage than those given by less similar users and so on. This problem can be solved by using a weighted average approach. In this approach, you multiply the rating of each user with a similarity factor calculated using the above mention formula. The missing rating can be calculated as

$$r_{up} = \bar{r}_u + \frac{\sum_{i \in users} sim(u,i) * r_{ip}}{\sum_{i \in users} |sim(u,i)|}$$

Implementation:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Sample User-Item Rating Matrix
data = {
    'Item1': [5, 4, 0, 1],
```

```
    'Item2': [3, 0, 0, 1],
    'Item3': [4, 0, 0, 1],
    'Item4': [4, 0, 0, 5],
    'Item5': [0, 1, 5, 4]
}

users = ['User1', 'User2', 'User3', 'User4']
df = pd.DataFrame(data, index=users)

print("User-Item Matrix:")
print(df)

# Step 1: Compute User Similarity (Cosine Similarity)
similarity_matrix = cosine_similarity(df)
similarity_df = pd.DataFrame(similarity_matrix, index=users, columns=users)

print("\nUser Similarity Matrix:")
print(similarity_df)

# Step 2: Predict Missing Ratings using Weighted Average

def predict_rating(user, item):
    user_index = df.index.get_loc(user)
    sim_scores = similarity_matrix[user_index]

    item_ratings = df[item].values

    numerator = (sim_scores * item_ratings).sum()
    denominator = abs(sim_scores).sum()

    if denominator == 0:
        return 0
```

```
    return numerator / denominator

# Example: Predict rating for User2 on Item3
predicted_rating = predict_rating('User2', 'Item3')

print("\nPredicted rating of User2 for Item3:", predicted_rating)

# Recommend items for User2 (items not rated = 0)
unrated_items = [item for item in df.columns if df.loc['User2', item] == 0]

recommendations = {}
for item in unrated_items:
    recommendations[item] = predict_rating('User2', item)

# Sort recommendations
recommended_items = sorted(recommendations.items(), key=lambda x: x[1], reverse=True)

print("\nRecommended Items for User2:")
for item, score in recommended_items:
    print(item, ":", score)
```

Conclusion:

In this experiment, User-Based Nearest Neighbor recommendation was implemented using collaborative filtering. The similarity between users was calculated using cosine similarity, which helped in identifying users with similar preferences. Based on these similarities, missing ratings were predicted using a weighted average approach. The system successfully recommended items to users by leveraging the preferences of similar users. This method is effective in personalized recommendation systems but may face challenges such as sparsity and scalability in large datasets.

Experiment No. 5
Implement Item-based Nearest neighbor recommendation
Date of Performance:
Date of Submission:
Marks:
Sign:

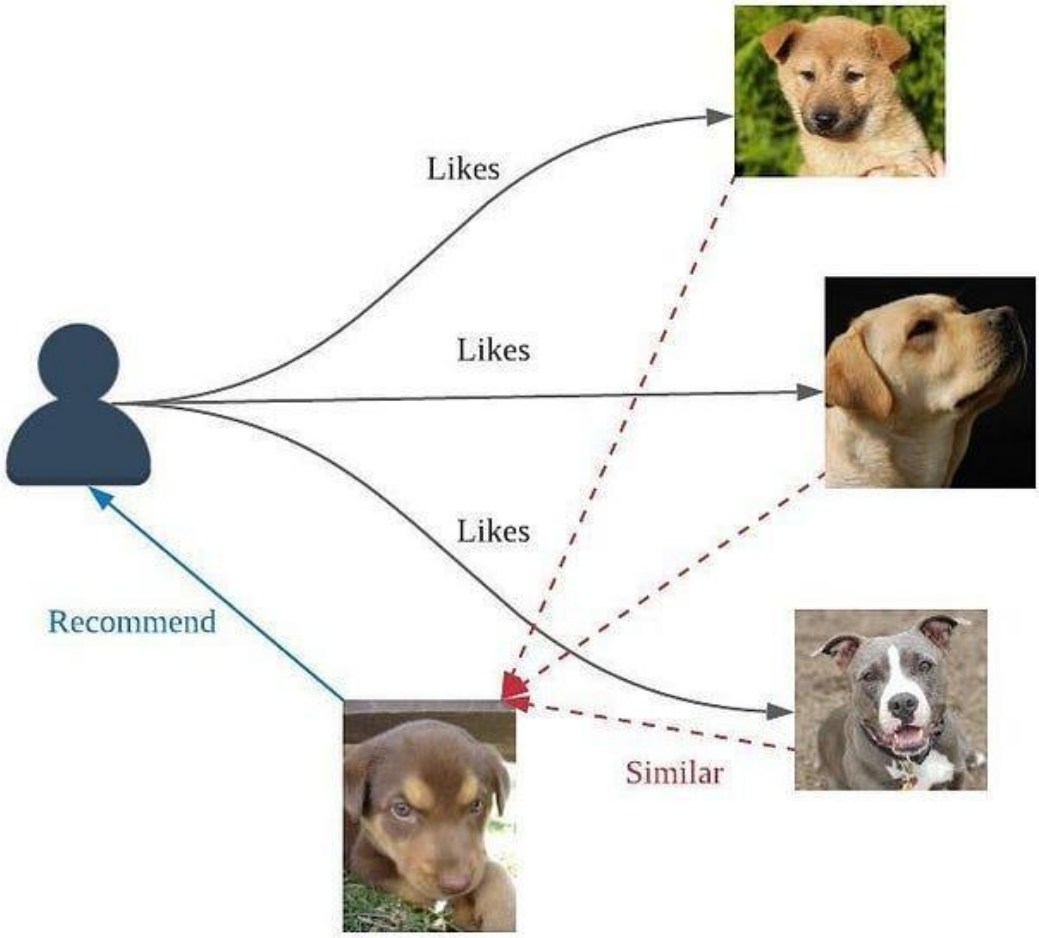
Aim: Implement Item-based Nearest neighbor recommendation.

Objective: Able to interpret and implement Item based Nearest neighbor recommendation.

Theory:

Item-based Collaborative Filtering

Item-based collaborative filtering uses the rating of co-rated item to predict the rating on specific item.



For example, we want to predict rating of user 2 on item 2.

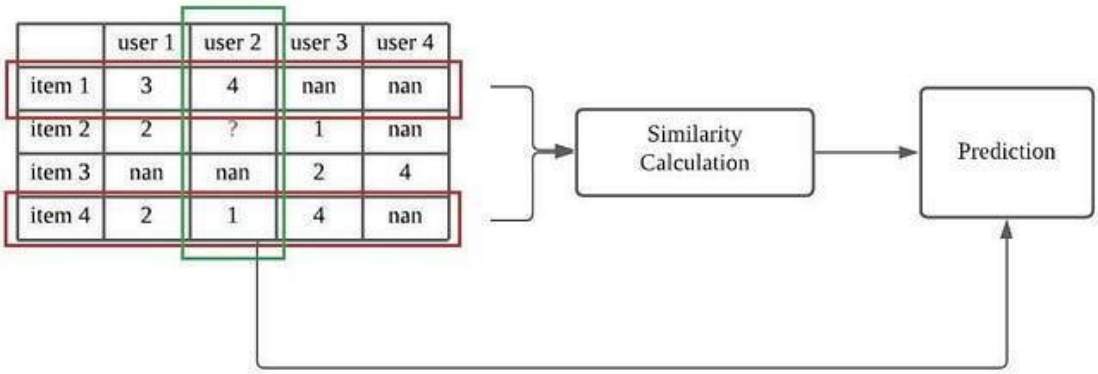
	user 1	user 2	user 3	user 4
item 1	3	4	nan	nan
item 2	2	?	1	nan
item 3	nan	nan	2	4
item 4	2	1	4	nan

To predict the rating,

1. Find the co-rated items of user 2, which is item 1 and item 4
2. Calculate the similarity between item 2 and item 1, 4
3. Calculate the prediction based on the similarity and co-rated rating, such that

$$prediction_{u,i} = \frac{\sum_n \omega_{i,n} * r_{u,n}}{\sum_n |\omega_{i,n}|}$$

where w is the similarity, and r is the rating value.



Implementation:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Sample User-Item Rating Matrix
data = {
    'Item1': [5, 4, 0, 1],
    'Item2': [3, 0, 0, 1],
    'Item3': [4, 0, 0, 1],
    'Item4': [4, 0, 0, 5],
    'Item5': [0, 1, 5, 4]
}

users = ['User1', 'User2', 'User3', 'User4']
df = pd.DataFrame(data, index=users)

print("User-Item Matrix:")
print(df)

# Step 1: Compute Item Similarity (Cosine Similarity)
item_similarity = cosine_similarity(df.T)
item_similarity_df = pd.DataFrame(item_similarity, index=df.columns,
columns=df.columns)

print("\nItem Similarity Matrix:")
print(item_similarity_df)

# Step 2: Predict Missing Ratings using Weighted Average

def predict_rating(user, target_item):
    user_ratings = df.loc[user]
```

```
    similarities = item_similarity_df[target_item]

    numerator = 0
    denominator = 0

    for item in df.columns:
        if user_ratings[item] > 0 and item != target_item:
            numerator += similarities[item] * user_ratings[item]
            denominator += abs(similarities[item])

    if denominator == 0:
        return 0
    return numerator / denominator

# Example: Predict rating for User2 on Item2
predicted_rating = predict_rating('User2', 'Item2')

print("\nPredicted rating of User2 for Item2:", predicted_rating)

# Recommend items for User2
unrated_items = [item for item in df.columns if df.loc['User2', item] == 0]

recommendations = {}
for item in unrated_items:
    recommendations[item] = predict_rating('User2', item)

# Sort recommendations
recommended_items = sorted(recommendations.items(), key=lambda x: x[1],
reverse=True)

print("\nRecommended Items for User2:")
```

```
for item, score in recommended_items:
    print(item, ":", score)
```

Conclusion:

In this experiment, Item-Based Nearest Neighbor recommendation was implemented using collaborative filtering. The similarity between items was calculated using cosine similarity, which helped identify items that are similar based on user ratings. Missing ratings were predicted using a weighted average of ratings from similar items. This approach provides more stable and scalable recommendations compared to user-based methods, especially when the number of users is large. Hence, item-based collaborative filtering is widely used in modern recommendation systems.

Experiment No. 6
Implement Content-based recommendation system
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implement Content-based recommendation system

Objective: Able to design and implement a content based recommendation system.

Theory:
Content-based filtering system: Content-Based recommender system tries to guess the features or behaviour of a user given the item's features, he/she reacts positively to.

Movies	User 1	User 2	User 3	User 4	Action	Comedy
Item 1	1		4	5	Yes	No
Item 2	5	4	1	2	No	Yes
Item 3	4	4		3	Yes	Yes
Item 4	2	2	4	4	No	Yes

The last two columns Action and Comedy Describe the Genres of the movies. Now, given these genres, we can know which users like which genre, as a result, we can obtain features corresponding to that particular user, depending on how he/she reacts to movies of that genre. Once, we know the likings of the user we can embed him/her in an embedding space using the feature vector generated and recommend him/her according to his/her choice. During recommendation, the similarity metrics (We will talk about it in a bit) are calculated from the item's feature vectors and the user's preferred feature vectors from his/her previous records. Then, the top few are recommended.
Content-based filtering does not require other users' data during recommendations to one user.

Implicit Feedback: The user's likes and dislikes are noted and recorded on the basis of his/her actions like clicks, searches, and purchases. They are found in abundance, but negative feedback is not found.

Explicit Feedback: The user specifies his/her likes or dislikes by actions like reacting to an item or rating it. It has both positive and negative feedback but less in number.

Implementation:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Sample Movie Dataset with Features (Content)
movies = pd.DataFrame({
    'movie_id': [1, 2, 3, 4, 5],
    'title': ['Movie1', 'Movie2', 'Movie3', 'Movie4', 'Movie5'],
    'Action': [1, 0, 1, 0, 1],
    'Comedy': [0, 1, 1, 0, 0]
})

print("Movies Dataset:")
print(movies)

# Sample User Ratings (Explicit Feedback)
ratings = pd.DataFrame({
    'movie_id': [1, 2, 3],
    'rating': [5, 3, 4]
})

print("\nUser Ratings:")
print(ratings)

# Step 1: Merge ratings with movie features
user_data = pd.merge(ratings, movies, on='movie_id')

# Step 2: Create User Profile (Weighted feature vector)
user_profile = {
    'Action': (user_data['Action'] * user_data['rating']).sum(),
```

```
    'Comedy': (user_data['Comedy'] * user_data['rating']).sum()
}

user_profile_df = pd.DataFrame([user_profile])

print("\nUser Profile (Feature Preferences):")
print(user_profile_df)

# Step 3: Compute similarity between user profile and all movies
movie_features = movies[['Action', 'Comedy']]

similarity_scores = cosine_similarity(user_profile_df, movie_features)

movies['similarity'] = similarity_scores[0]

# Step 4: Recommend movies (excluding already rated)
recommended_movies = movies[~movies['movie_id'].isin(ratings['movie_id'])]
recommended_movies = recommended_movies.sort_values(by='similarity',
ascending=False)

print("\nRecommended Movies:")
print(recommended_movies[['title', 'similarity']])
```

Conclusion:

In this experiment, a Content-Based Recommendation System was implemented using Python. The system utilized item features such as genres (Action and Comedy) along with user ratings to build a user profile representing individual preferences. Cosine similarity was used to measure the similarity between the user profile and movie features, enabling personalized recommendations. Unlike collaborative filtering, this method does not rely on other users' data and works effectively even with a single user's history. Thus, content-based filtering provides accurate and personalized recommendations based on user interests.

Experiment No. 7
Implementation of a recommendation system using Hybrid approach
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implementation of a recommendation system using Hybrid approach

Objective: Able to implement and understand the aspects of Hybrid Recommendation System.

Theory:

Hybrid recommendation systems bring the best of both worlds together. Hybrid mechanisms to predict recommendation, as the name implies, combine two or more recommendation techniques. The disadvantages of a recommendation methodology can be resolved using hybrid technology, and the benefits of various recommendation techniques can be summed up. The hybrid recommendation methodology to develop a search model for selecting the best cloud service provider. It recommended service providers close to the organization to leverage the enterprise location to boost bandwidth and eliminate latency difficulties.

In addition, this work proposes a search model, recommendation system and evaluation standard for customers based on their needs and location. According to the implementation results, cloud servers must be located close to the enterprise in order to improve bandwidth and reduce latency. This framework tracks the enterprise's location, and the recommended function recommends the closest cloud server to the enterprise. Two models are used to classify the essential factors used for the searching function: the basic and feature models. These models can make it easier for enterprises to find what they need in the system. In, the author discusses how recommendation systems play an important role in providing adequate information to users and filtering data. The various recommendation techniques, such as Collaborative Filtering, demographic-based, and knowledge-based, all have drawbacks. While opting for recommendation systems, these techniques fail to provide effective recommendations. As a result, it is necessary to optimize these techniques by identifying more distinct features. This is possible by integrating the advantages of various techniques in a hybrid manner. The author proposed an effective hybrid technique for book recommendations that uses ontology for user profiling to improve system efficiency. The results show that combination of Collaborative Filtering, knowledge-based techniques, and demographic attributes into a Hybrid technique produces the best recommendations.

A hybrid recommendation, the framework uses a combination of content-based recommendation system(CBRS) and collaborative filtering recommendation systems (CFRS) to achieve precise performance by reducing the drawbacks of the conventional recommendation techniques.

Implementation:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# -------------------------------
```

```
# Step 1: User-Item Ratings Matrix (Collaborative Filtering)
# -------------------------------
ratings = pd.DataFrame({
    'Item1': [5, 4, 0, 1],
    'Item2': [3, 0, 0, 1],
    'Item3': [4, 0, 0, 1],
    'Item4': [4, 0, 0, 5],
    'Item5': [0, 1, 5, 4]
}, index=['User1', 'User2', 'User3', 'User4'])

# -------------------------------
# Step 2: Item Features (Content-Based)
# -------------------------------
features = pd.DataFrame({
    'Item': ['Item1', 'Item2', 'Item3', 'Item4', 'Item5'],
    'Action': [1, 0, 1, 0, 1],
    'Comedy': [0, 1, 1, 0, 0]
}).set_index('Item')

# -------------------------------
# Step 3: Collaborative Filtering (User Similarity)
# -------------------------------
user_similarity = cosine_similarity(ratings)
user_similarity_df = pd.DataFrame(user_similarity, index=ratings.index,
columns=ratings.index)

# Predict rating using User-Based CF
def cf_predict(user, item):
    sim_scores = user_similarity_df[user]
    item_ratings = ratings[item]

    numerator = (sim_scores * item_ratings).sum()
```

```
    denominator = sim_scores.sum()

    if denominator == 0:
        return 0
    return numerator / denominator

# -------------------------------
# Step 4: Content-Based Filtering
# -------------------------------
# Create user profile
def get_user_profile(user):
    user_ratings = ratings.loc[user]
    profile = (features.T * user_ratings).T.sum()
    return profile

# Predict using content similarity
def cb_predict(user, item):
    user_profile = get_user_profile(user).values.reshape(1, -1)
    item_vector = features.loc[item].values.reshape(1, -1)

    similarity = cosine_similarity(user_profile, item_vector)[0][0]
    return similarity

# -------------------------------
# Step 5: Hybrid Recommendation (Combine CF + CB)
# -------------------------------
def hybrid_score(user, item, alpha=0.5):
    cf_score = cf_predict(user, item)
    cb_score = cb_predict(user, item)

    return alpha * cf_score + (1 - alpha) * cb_score
```

```
# Recommend items for a user
user = 'User2'
unrated_items = [item for item in ratings.columns if ratings.loc[user, item] == 0]

recommendations = {}
for item in unrated_items:
    recommendations[item] = hybrid_score(user, item)

# Sort recommendations
recommended_items = sorted(recommendations.items(), key=lambda x: x[1],
reverse=True)

print("Hybrid Recommendations for", user)
for item, score in recommended_items:
    print(item, ":", score)
```

Conclusion

In this experiment, a Hybrid Recommendation System was successfully implemented by combining Collaborative Filtering and Content-Based Filtering techniques. The collaborative approach utilized user similarity to predict preferences, while the content-based approach leveraged item features to understand user interests. By integrating both methods, the hybrid system effectively reduced the limitations of individual techniques, such as cold start and sparsity problems. The combined model provided more accurate and personalized recommendations, demonstrating that hybrid approaches are highly effective in real-world recommendation systems.

Experiment No. 8
Implementation of a Regression based recommendation system
Date of Performance:
Date of Submission:
Marks:
Sign:

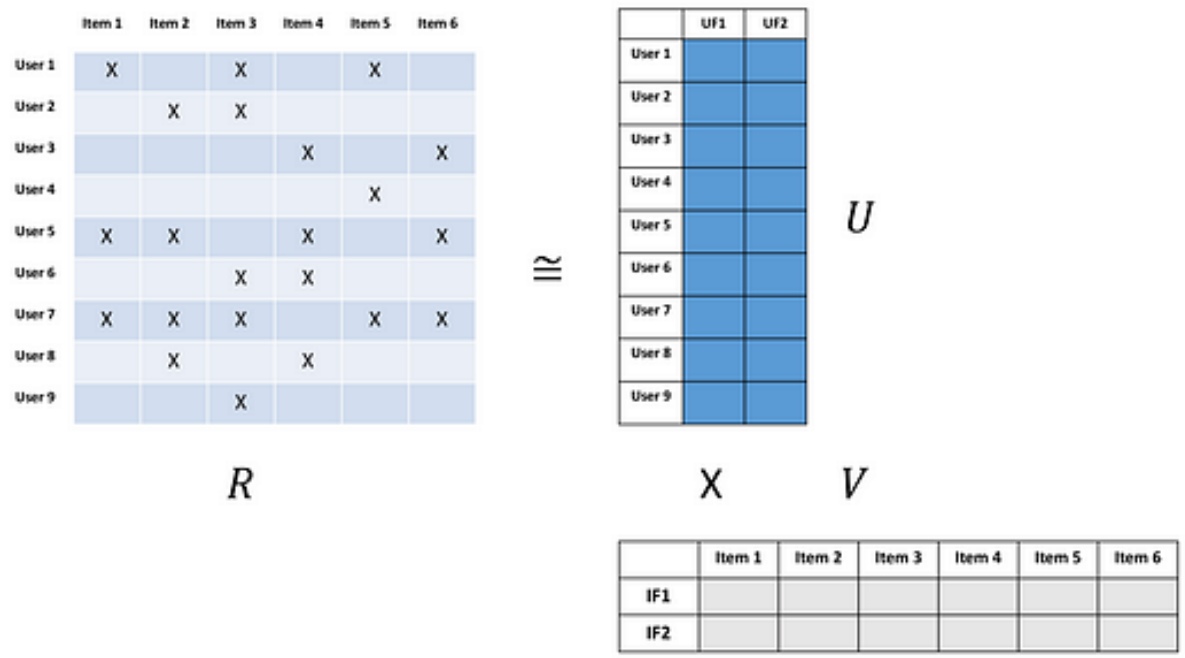
Aim: Implementation of a Regression based recommendation system.

Objective: Able to design a regression based recommendation system with respective implementation.

Theory: Often, matrix factorization is applied in the realm of dimensionality reduction, where we are trying to reduce the number of features while still keeping the relevant information. This is the case with principal component analysis (PCA) and the very similar singular value decomposition (SVD).

Essentially, can we take a large matrix of user/item interactions and figure out the latent (or hidden) features that relate them to each other in a much smaller matrix of user features and item features? That's exactly what ALS is trying to do through matrix factorization.

As the image below demonstrates, let's assume we have an original ratings matrix RR of size MxNMxN, where MM is the number of users and NN is the number of items. This matrix is quite sparse, since most users only interact with a few items each. We can factorize this matrix into two separate smaller matrices: one with dimensions MxKMxK which will be our latent user feature vectors for each user (U)(U) and a second with dimensions KxNKxN, which will have our latent item feature vectors for each item (V)(V). Multiplying these two feature matrices together approximates the original matrix, but now we have two matrices that are dense including a number of latent features KK for each of our items and users.



In order to solve for UU and VV, we could either utilize SVD (which would require inverting a potentially very large matrix and be computationally expensive) to solve the factorization more precisely or apply ALS to approximate it. In the case of ALS, we only need to solve one feature vector at a time, which means it can be run in parallel! (This large advantage is probably why it is the method of choice for Spark). To do this, we can randomly initialize UU and solve for VV. Then we can go back and solve for UU using our solution for VV.

Implementation:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Sample User-Item Ratings Data
data = {
   'user_id': [1, 1, 2, 2, 3, 3, 4, 4],
   'item_id': [1, 2, 1, 3, 2, 4, 3, 4],
   'rating':  [5, 3, 4, 2, 4, 5, 3, 4]
}

df = pd.DataFrame(data)

print("Dataset:")
print(df)

# -------------------------------
# Step 1: Create Features
# -------------------------------
# Using user_id and item_id as features
X = df[['user_id', 'item_id']]
y = df['rating']
```

```
# -------------------------------
# Step 2: Train Regression Model
# -------------------------------
model = LinearRegression()
model.fit(X, y)

# -------------------------------
# Step 3: Predict Ratings
# -------------------------------
# Predict for all user-item pairs
users = df['user_id'].unique()
items = df['item_id'].unique()

predictions = []

for user in users:
    for item in items:
        pred_rating = model.predict([[user, item]])[0]
        predictions.append([user, item, pred_rating])

pred_df = pd.DataFrame(predictions, columns=['user_id', 'item_id',
'predicted_rating'])

print("\nPredicted Ratings:")
print(pred_df)

# -------------------------------
# Step 4: Recommend Items
# -------------------------------
# Recommend top items for a specific user (e.g., User 2)
user_id = 2
```

```
# Remove already rated items
rated_items = df[df['user_id'] == user_id]['item_id'].tolist()

recommendations = pred_df[(pred_df['user_id'] == user_id) &
                          (~pred_df['item_id'].isin(rated_items))]

# Sort by predicted rating
recommendations = recommendations.sort_values(by='predicted_rating',
ascending=False)

print("\nRecommended Items for User", user_id)
print(recommendations)
```

Conclusion:

In this experiment, a Regression-Based Recommendation System was implemented using a Linear Regression model. The system predicted user-item ratings based on features such as user ID and item ID. By learning patterns from existing ratings, the model was able to estimate unknown ratings and recommend items accordingly. Although simple, regression-based methods provide a foundational understanding of predictive recommendation systems. However, they may not capture complex user-item relationships as effectively as advanced techniques like matrix factorization or deep learning models.